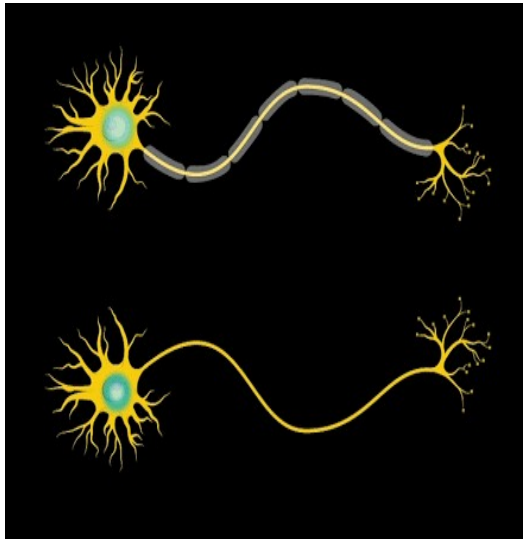
Laboratório 06:

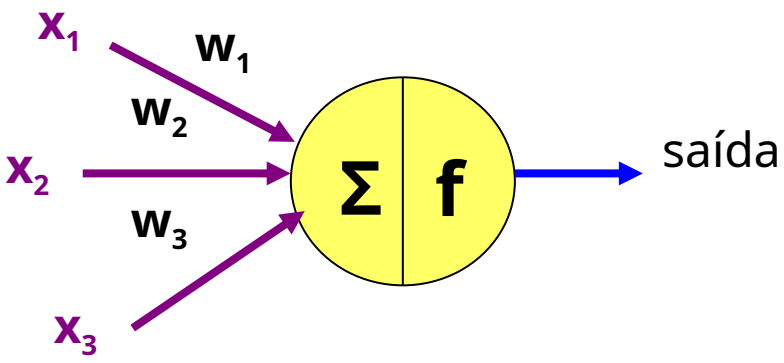
Multi-Layer Perceptron

CC7711 - Inteligência Artificial

Prof. Dr. Murillo | Depto. de Ciência da Computação

Perceptron

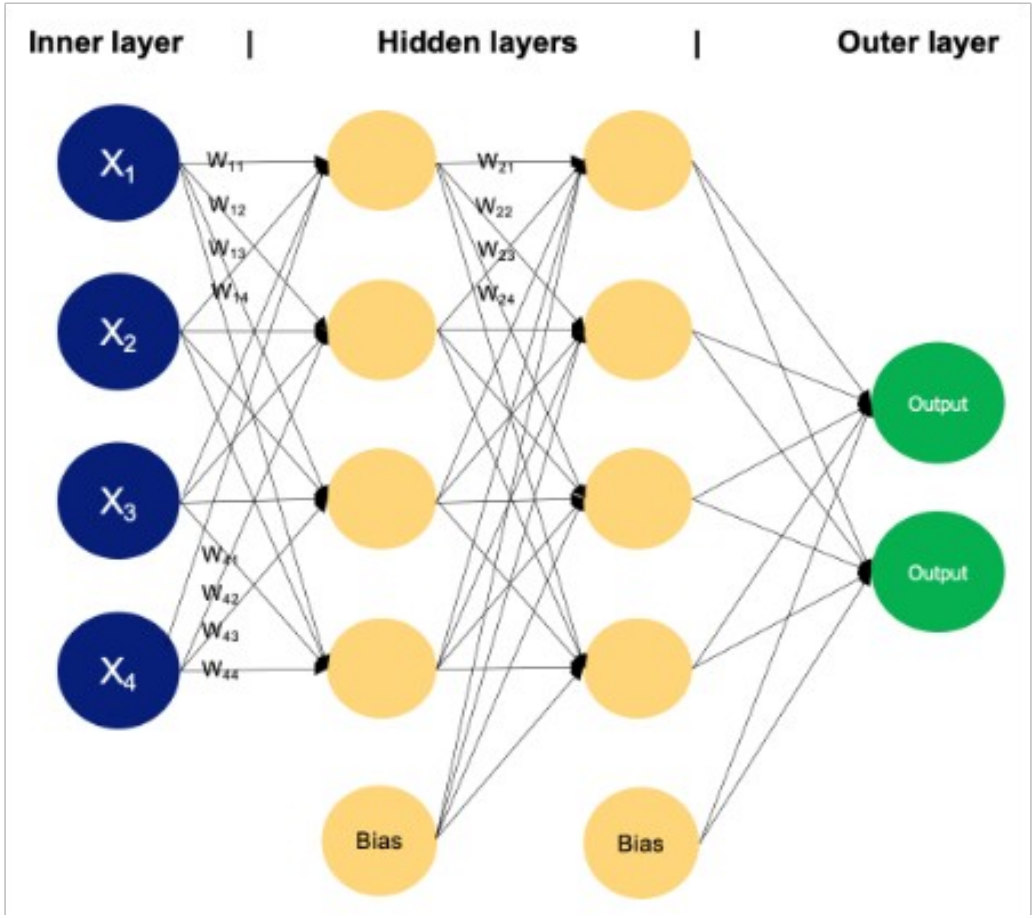




- **Xi:** Vetor de entradas.
- **Wi:** Vetor de pesos.
- **f:** Função de ativação.
- **saída** = $f(w^t.x)$

Multi-Layer Perceptron (MLP)

- **Input:** Recebe os dados de entrada.
- **Hidden:** Aprende os padrões de classificação
- **Saída:** Retorna a classificação final



Matriz de Confusão

Tabela que compara os valores **Reais vs Predito** pelo modelo

- Verdadeiros Positivos (VP)
- Falsos Negativos (FN)
- Falsos Positivos (FP)
- Verdadeiros Negativos (VN)

		Predicted Class: Positive	Predicted Class: Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	**Sensitivity** $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	**Specificity** $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	**Negative Predictive Value** $\frac{TN}{(TN + FN)}$	**Accuracy** $\frac{TP + TN}{(TP + TN + FP + FN)}$

Scikit-learn

https://scikit-learn.org/

```
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(10, 10),
max_iter=100, random_state=42)

mlp.fit(Xtrain, ytrain)
```

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_estimator(mlp, Xtest, ytest,
display_labels=target_names)
plt.title('Matriz de Confusão')
plt.show()
```

Desafio 1: IRIS Dataset

Utilizar MLP para classificar diferentes tipos de flores da espécie Iris :

Iris Versicolor **Iris Setosa** **Iris Virginica**

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names
```

Desafio 2: MNIST Dataset

Utilizar MLP para reconhecer diferentes digitos da base de dados MNIST

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X_mnist, y_mnist = mnist.data, mnist.target
X_mnist = X_mnist / 255.0 #normalização
```

Desafio 3: Human Activity Recognition with Smartphones

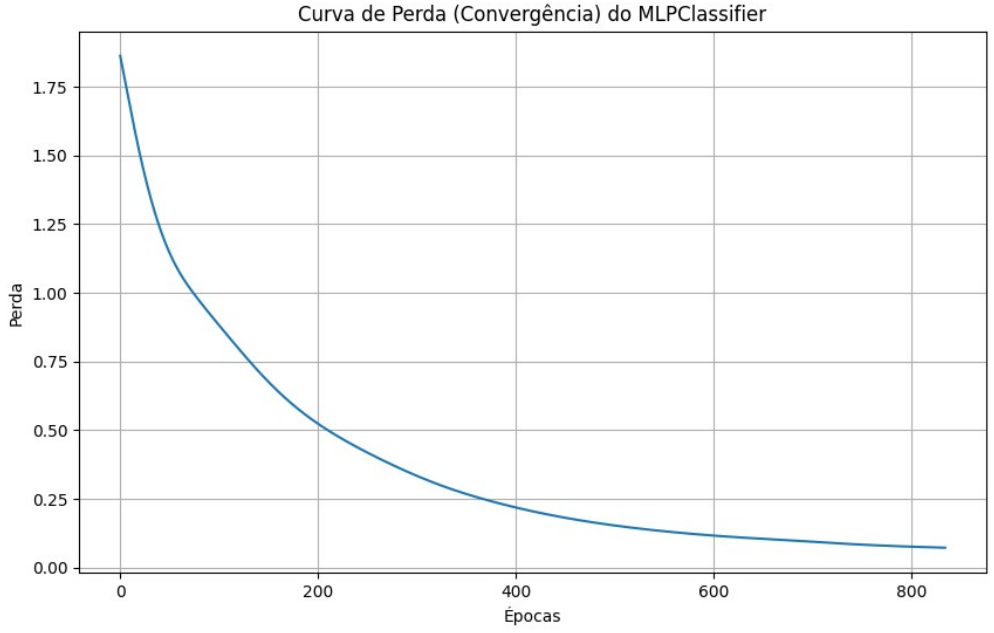
Utilizar MLP para classificar a atividade humana de uma pessoa enquanto ela utilizava o celular a partir do dataset [Human Activity Recognition with Smartphones](#).

Utilizar LabelEncoder() e StandarScaler() do scikit-learn para realizar o treinamento.

Métricas e Convergência

Monitorar o valor de fitness para cada iteração do treinamento.

- **Gráfico de Convergência:** Evolução do Melhor Fitness vs Épocas.
- **Early Stopping:** Encerra o treinamento se o valor de fitness não tem mais melhorias.
- **Classification Report:** Calcula as principais métricas de avaliação do modelo treinado.




```
from sklearn.metrics import classification_report

y_pred = mlp.predict(Xtest)
print(classification_report(ytest, y_pred))

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(mlp.loss_curve_)
plt.show()
```

Roteiro do Laboratório & Entregas

Utilizem o template de código Python (.ipynb) fornecido para completar as seguintes tarefas:

- **Passo 1:** Classificar a base da Iris utilizando MLP
- **Passo 2:** Classificar a base da MNIST utilizando MLP
- **Passo 3:** Classificar a base Human Activity Recognition with Smartphones utilizando MLP

Métricas e Submissão: Submetam o notebook com o código completo. Ao final, apresentem a solução dos 3 problemas com as métricas: **Classification Report, Matriz de Confusão e Gráfico de Convergência.**
(Observação: obter uma acurácia de pelo menos 90%)